

Aegis AI — Link Analysis Service

Complete API Documentation

Version 4.0 | Final Year Project (FYP)

Table of Contents

1. What Is Aegis?
 2. How It Works (Non-Technical)
 3. Architecture Overview
 4. Detection Layers
 5. Risk Classification System
 6. Machine Learning Classifier
 7. API Reference
 8. Scoring System (Technical)
 9. Data Storage & Privacy
 10. Performance & Reliability
 11. Configuration Reference
 12. Limitations & Known Issues
 13. FYP Summary
-

1. What Is Aegis?

Aegis is a URL threat analysis API. You send it a link, it tells you whether that link is safe, suspicious, or dangerous — and exactly why.

The core problem it solves: When you receive a link in an email, message, or document, it is often impossible to tell whether it is safe just by looking at it. Attackers create URLs that look legitimate but steal your credentials, install malware, or redirect you to scam pages. Aegis automatically investigates the link across 11 different intelligence layers and produces a risk score with a full explanation.

Who it is built for: - End users who want to verify a suspicious link before clicking - Developers building phishing detection into applications (chatbots, email filters, browser extensions) - Security teams that need automated URL triage - Researchers studying phishing and malware distribution patterns

What makes it different from simply checking VirusTotal: VirusTotal checks the URL against antivirus databases — but only once the URL is already known and flagged. Aegis catches URLs that have *never been seen before* by

running its own structural, DNS, WHOIS, SSL, and redirect analysis. A brand-new phishing URL that has zero VirusTotal detections will still score high in Aegis because of its domain name, certificate, and HTTP redirect behaviour.

2. How It Works (Non-Technical)

When you submit a URL like `http://paypal-secure-verify-account.tk/login/confirm`, Aegis does the following:

Step 1 — Read the URL itself. The link uses HTTP (not HTTPS, so your data would be sent unencrypted). The domain contains “paypal”, “secure”, “verify”, “account” — all classic words attackers use to look trustworthy. The .tk ending is a free domain used overwhelmingly for fraud. These observations alone raise the alarm.

Step 2 — Look up the domain’s history. Aegis checks WHOIS records to find out when the domain was registered and who owns it. Legitimate organizations register domains years in advance. Phishing domains are usually registered days before an attack. If registration information is hidden or unavailable, that is itself a red flag.

Step 3 — Check if the domain even works properly. A legitimate company always has a working mail server, matching DNS records, and proper name servers. Aegis checks all of these. A dead or misconfigured domain that cannot receive email is a strong phishing indicator.

Step 4 — Verify the security certificate. HTTPS websites use certificates to prove their identity. Aegis checks whether the certificate is valid, who issued it, how old it is, and whether it matches the domain. Phishing sites often use free automated certificates on very new domains.

Step 5 — Follow any redirects. Attackers often hide their real destination behind redirect chains. A link might appear to go to one place but redirect through URL shorteners and finally land on a malicious page. Aegis follows the entire chain and reports every hop.

Step 6 — Check threat databases. Aegis queries three live databases: URLhaus (known malware URLs), OpenPhish (known phishing URLs), and Google Safe Browsing (Google’s own blocklist). If the URL appears in any of these, it is immediately flagged.

Step 7 — Run it through 94 antivirus engines. Via VirusTotal, the URL is checked against every major antivirus and threat intelligence provider simultaneously.

Step 8 — Apply a machine learning model. A Random Forest classifier trained on thousands of URLs makes its own independent prediction, looking

at all 35 extracted features from the URL and its scan results.

Step 9 — Calculate the final risk score. All signals are combined with weighted scoring. Critical signals (3+ antivirus detections, confirmed phishing feed match) can override the mathematical result to ensure dangerous URLs are always classified as High Risk.

Step 10 — Return a full report. The response includes the risk level, confidence percentage, all flags raised, score breakdown by category, and the ML prediction with explanation of which features contributed.

3. Architecture Overview

Client Request

FastAPI Application (Python 3.11, Uvicorn ASGI)

Redis (Caching layer - 1 hour TTL)
Returns instantly on cache hit

Heuristics Engine (synchronous, CPU-bound)

Concurrent Layer	
WHOIS Analysis	(python-whois)
DNS Intelligence	(dnspython)
SSL/TLS Inspection	(ssl + httpx)
Redirect Chain Tracing	(httpx async)
URLhaus Lookup	(abuse.ch API)
OpenPhish Feed Check	(cached in Redis, 6h TTL)
Google Safe Browsing	(GSB API v4)

External APIs (concurrent)
VirusTotal submission + polling (94 engines)
URLScan.io submission + polling (visual scan)

ML Classifier (scikit-learn Random Forest)

Risk Classification Engine (weighted scoring + overrides)

Pinecone Vector DB (semantic memory / similarity search)

SQLite (Feedback storage, persistent across restarts)

Technology Stack: | Component | Technology | Purpose | |———| |———| |———|

——| | API Framework | FastAPI 0.109 | High-performance async Python API
 | | Server | Uvicorn (ASGI) | Async server for concurrent requests | | Caching |
 Redis | Scan result cache, OpenPhish feed cache, rate limit state | | ML Runtime |
 scikit-learn 1.4 | Random Forest classifier inference | | Vector Memory | Pinecone
 | Semantic similarity search across past scans | | Feedback Store | SQLite +
 aiosqlite | Persistent labeled training data | | HTTP Client | httpx | Async
 HTTP for all external calls | | Containerization | Docker + Docker Compose |
 Reproducible deployment |

4. Detection Layers

Layer 1 — URL Heuristics (14 checks)

Analyses the URL string itself without making any network requests. Runs in
 < 1ms.

Check	What It Detects	Score Contribution
Suspicious TLD	.tk, .ml, .xyz, .top and 28 others used predominantly for fraud	+20
Typosquatting	Brand names (paypal, google, amazon, apple, microsoft) embedded in non-official domains	+25
High entropy	Shannon entropy > 3.5 suggests randomly-generated domain names (botnet/malware pattern)	+15
Phishing keywords	login, verify, secure, account, confirm, password, billing etc. in URL	+20
HTTP protocol	No HTTPS = data is transmitted unencrypted	+10
IP address host	URLs using raw IP addresses instead of domain names	+15
@ symbol	https://google.com@evil.com — browser navigates to evil.com	+20
Excessive length	URL > 100 characters	+5

Check	What It Detects	Score Contribution
Port number	Non-standard ports (e.g., :8080, :8443)	+10
Double slash	Malformed path // used to confuse parsers	+5
Hex encoding	%68%74%74%70 style encoding hiding URL content	+10
Multiple subdomains	login.secure.paypal.evil.com style nesting	+10
Redirect parameter	?redirect=, ?url=, ?go= suggesting open redirect	+15
URL shortener	bit.ly, tinyurl, goo.gl — destination hidden	+15

Layer 2 — WHOIS Domain Age Analysis

Queries the global WHOIS database to find when the domain was registered and who owns it.

What it checks: - Domain creation date and age (days since registration) - Registration expiry date and period length - Registrar name (some registrars are disproportionately used for fraud) - Country of registration - Whether registration information is hidden (privacy-shielded)

Why it matters: Phishing campaigns are typically launched within days of domain registration. A domain registered yesterday claiming to be “PayPal’s secure verification portal” is almost certainly fraudulent. Legitimate companies own their domains for years.

Score thresholds: - Domain < 30 days old: +40 to WHOIS score - Domain < 90 days old: +20 - WHOIS information hidden/unavailable: +15 - Short registration period (< 1 year): +10 - Known abusive registrar: +10

Layer 3 — DNS Intelligence

Queries DNS infrastructure to verify the domain’s internet presence is consistent with a legitimate organization.

What it checks: - A records: Does the domain resolve to an IP address? - MX records: Does the domain have mail servers configured? - SPF records: Does the domain have email anti-spoofing configured? - DMARC: Does the domain have email authentication policies? - CNAME chain depth: Long CNAME chains can indicate redirection infrastructure - Nameservers: Are custom name servers configured, or free/shared ones? - Reverse DNS: Does the IP address point back to the domain?

Why it matters: Phishing domains are often hastily configured and lack the full DNS infrastructure of legitimate businesses. A domain claiming to be a major bank but with no mail servers and a single generic nameserver is extremely suspicious.

Layer 4 — SSL/TLS Certificate Analysis

Inspects the HTTPS certificate for the domain.

What it checks: - Certificate validity (not expired, not self-signed) - Certificate issuer (major trusted CA vs. free automated CA) - Certificate age (brand-new certs on new domains = phishing risk) - Days until expiry (very short = suspicious) - TLS protocol version (TLSv1.3 is modern/good; TLSv1.0/1.1 is outdated) - Common Name match (does the cert actually cover this domain?) - Subject Alternative Names (how many domains does this cert cover?)

Important nuance: Let's Encrypt is a legitimate free Certificate Authority used by many real websites including Claude.ai. The system penalises free CAs with a small score penalty because phishing sites disproportionately use them — but this alone cannot classify a URL as malicious. The final classification uses weighted scoring across all layers.

Layer 5 — Redirect Chain Tracing

Follows the full HTTP redirect chain from the submitted URL to the final destination.

What it checks: - Total number of redirect hops - Whether any hop is a known URL shortener (bit.ly, tinyurl, t.co, etc.) - Protocol downgrade (HTTPS → HTTP anywhere in the chain = dangerous) - Destination domain change (final domain differs from submitted domain) - WWW normalization (google.com → www.google.com is normal, not suspicious) - Final destination status code (404 = broken/taken-down phishing page) - Connection failures (domain is dead or blocking requests)

Layer 6 — URLhaus Malware Feed

Queries abuse.ch's URLhaus database — a community-maintained list of active malware distribution URLs.

- Updated continuously by security researchers worldwide
- Covers malware downloads, botnet C&C servers, exploit kit landing pages
- Requires free API registration at <https://auth.abuse.ch>

Match types: - Exact URL match: Score = 100 (confirmed malware) - Domain match: Score = 80 (domain hosts other malware)

Layer 7 — OpenPhish Phishing Feed

Checks against OpenPhish’s live phishing URL database. Updated every 6 hours and cached in Redis to avoid fetching on every request.

- Community-maintained list of confirmed phishing URLs
- ~50,000 live phishing pages in the commercial feed (public feed ~300–500)
- No API key required

Match types: - Exact URL match: Score = 100 (confirmed phishing) - Domain match: Score = 75 (domain hosts other phishing pages)

Layer 8 — Google Safe Browsing

Queries Google’s Safe Browsing API, which powers the malicious URL warnings in Chrome, Firefox, and Safari.

- Covers phishing, malware, unwanted software, social engineering
- Updated in real-time by Google’s global threat detection infrastructure
- Free API with rate limits; requires Google Cloud API key

Layer 9 — VirusTotal (94 Antivirus Engines)

Submits the URL to VirusTotal, which checks it simultaneously against 94 antivirus engines and threat intelligence providers including Kaspersky, BitDefender, Malwarebytes, Trend Micro, and more.

Detection thresholds: - 1–2 malicious detections: Low-risk signal (may be false positive) - 3–5 malicious detections: Critical signal — triggers High Risk override - 5+ malicious detections: Extreme confidence — 90+ score guaranteed

Note: Results take 60–90 seconds as VirusTotal processes the submission asynchronously. The Aegis API polls until results are ready.

Layer 10 — URLScan.io Visual Analysis

Submits the URL to URLScan.io, which loads the page in a real browser, takes a screenshot, and analyses the DOM, JavaScript, cookies, and network requests.

What it provides: - Full-page screenshot (now with US English locale — German screenshots were a bug, now fixed) - Detailed page load analysis - JavaScript behaviour analysis - Network traffic analysis - Link to complete report on URLScan’s website

Layer 11 — ML Classifier (Random Forest)

An independent prediction from a machine learning model trained on thousands of labeled URLs.

How it works: See Section 6 (Machine Learning Classifier) for full details.

5. Risk Classification System

Risk Levels

Level	Score Range	Meaning
Safe	0–14.99	No significant threat indicators. Highly confident this URL is not malicious.
Low Risk	15–34.99	Minor suspicious signals. Likely safe but worth checking. Probably a false positive or mildly suspicious domain.
Medium Risk	35–64.99	Multiple suspicious signals. Treat with caution. Do not enter credentials or download files unless you can verify the source through other means.
High Risk	65–100	Strong indicators of malicious intent. Do not visit this URL.

Weighted Scoring Formula

```
combined_score = (  
    vt_score      × 0.30 + # VirusTotal - highest weight, most reliable  
    heuristic_score × 0.15 + # URL structure analysis  
    whois_score   × 0.15 + # Domain age and registration  
    dns_score     × 0.08 + # DNS infrastructure  
    ssl_score     × 0.07 + # Certificate analysis  
    redirect_score × 0.10 + # Redirect chain (increased from 0.05 -  
Phase 3 fix)  
    urlhaus_score × 0.07 + # Malware feed  
    openphish_score × 0.05 + # Phishing feed  
    gsb_score     × 0.03   # Google Safe Browsing  
)
```


Critical Signal Overrides

Mathematical scoring alone can underclassify obvious threats. These hard overrides ensure dangerous URLs are always caught:

Condition	Override	Reason
VT malicious 5	Score 90	5+ antivirus detections = extremely high confidence
VT malicious 3	Score 65	Industry-standard threshold for confirmed malicious
VT malicious 1 + suspicious 2	Score 65	Multiple independent signals agree
3+ critical signals triggered together	Score 80	Multi-layer agreement = very high confidence
2 critical signals triggered	Score 45	Medium-high confidence threshold
OpenPhish/URLhaus/GSB match	Score 90	Confirmed by human-reviewed database
redirect_score 50 + any VT malicious	Score 38	Dangerous redirect with confirmed detection
redirect_score 70 alone	Score 25	Highly suspicious redirect behaviour

6. Machine Learning Classifier

Overview

The ML classifier is an independent prediction layer that complements the rule-based scoring system. It analyses the same URL characteristics but uses a trained statistical model rather than hand-written thresholds.

Model type: Random Forest Classifier with Platt scaling calibration **Features:** 35 numeric features extracted from scan results **Prediction:** Binary — 0 (benign) or 1 (phishing/malicious) **Output:** Calibrated probability (e.g., 85.27% phishing probability)

The 35 Features

URL Structure (Features 0–7)

Feature	Description	Example
url_length	URL length / 200, capped at 1.0	paypal-secure-verify.tk/login = 0.26
subdomain_depth	Subdomains / 5	login.secure.evil.com = 0.4
has_ip_address	1 if host is a raw IP	http://192.168.1.1/admin = 1.0
is_http	1 if URL uses HTTP	http:// = 1.0
suspicious_tld	1 if TLD in abusive list	.tk, .ml, .xyz = 1.0
entropy	Shannon entropy of domain / 5	Random domain = high
has_phishing_keywords	Fraction of keywords in URL	login+verify+secure = 0.6
has_at_symbol	1 if @ in URL	evil.com@legit.com = 1.0

WHOIS Signals (Features 8–11)

Feature	Description
domain_age_normalized	1 - age/3650. Newer domains score higher.
whois_unavailable	1 if WHOIS returned no creation date
registrar_abusive	1 if registrar in known-abusive list
short_registration	1 if registration period < 365 days

DNS Signals (Features 12–17)

Feature	Description
dns_no_resolve	1 if domain returns NXDOMAIN
dns_no_mx	1 if no MX records
dns_no_spf	1 if no SPF TXT record
dns_cname_depth	CNAME chain length / 5
dns_single_ns	1 if only one nameserver
dns_free_provider	1 if using free DNS (no-ip.com, dyndns)

SSL Signals (Features 18–23)

Feature	Description
ssl_invalid	1 if cert is invalid or missing
ssl_new_cert	1 if cert < 30 days old AND non-major CA
ssl_expiring_soon	1 if cert expires in < 30 days
ssl_free_ca	1 if issued by Let's Encrypt or similar
ssl_self_signed	1 if self-signed certificate
ssl_cn_mismatch	1 if CN doesn't match the hostname

Redirect Signals (Features 24–28)

Feature	Description
redirect_hops	Hop count / 10
has_shortener	1 if URL shortener in chain
protocol_downgrade	1 if HTTPS → HTTP in chain
destination_changed	1 if final domain differs (non-www)
final_404	1 if final destination is a 404

External API Signals (Features 29–34)

Feature	Description
vt_malicious_normalized	VT malicious count / 10
vt_suspicious_normalized	VT suspicious count / 10
urlhaus_hit	1 if found in URLhaus
openphish_hit	1 if found in OpenPhish
gsb_hit	1 if found in Google Safe Browsing
total_flags_normalized	Total flag count / 20

Training

Dataset: 10,000+ labeled URLs combining OpenPhish, URLhaus, and Alexa top-1M benign sites.

Architecture choices: - **Random Forest (300 trees):** Ensemble method — more robust than single decision tree, naturally handles non-linear relationships between features - **max_depth=8:** Prevents memorization (100% training accuracy = overfitting, fixed in v2.0) - **min_samples_leaf=10:** Requires 10 samples per leaf — prevents the model from learning from noise - **CalibratedClassifierCV (Platt scaling):** Ensures probabilities are meaningful — an 85% prediction really means ~85%, not 95% or 60% - **SMOTE:** Synthetic Minority Oversampling handles class imbalance in training data - **Proper 3-way split:** 60% train / 20% validation / 20% test — test set never seen during training

Expected performance: 88–93% accuracy on held-out test set. 100% accuracy during training = overfitting.

Why ML and Rule-Based Together

Neither system alone is sufficient:

- **Rules only:** Cannot adapt to new attack patterns, miss novel phishing techniques
- **ML only:** Opaque “black box”, harder to explain to examiner/client, requires training data

Together, they provide: 1. Explainable rule-based reasoning (flags, score breakdown) 2. Statistical pattern recognition across combinations of features 3. Independent confirmation — if both systems agree it’s dangerous, confidence is very high 4. When they disagree, the rule-based score takes precedence (more auditable)

7. API Reference

Base URL: `http://localhost:8000` **Interactive Documentation:** `http://localhost:8000/docs` **Content-Type:** `application/json` for all requests and responses

POST /scan

Submit a single URL for full threat analysis.

Request:

```
{  
  "url": "https://example.com"  
}
```

Note: URL scheme is optional — `google.com` is automatically normalized to `https://google.com`.

Rate limit: 5 requests per minute

Response (200 OK):

```
{  
  "url": "https://example.com",  
  "risk_level": "Safe",  
  "confidence_score": 1.2,  
  "message": "SAFE: The link shows no significant threat indicators. (98.8% confidence it
```

```

"scan_date": "2026-02-28",
"scan_id": "u-abc123...",
"detection_counts": {
  "malicious": 0, "suspicious": 0, "undetected": 20, "harmless": 74
},
"scanners_count": 94,
"virustotal_report": "https://www.virustotal.com/gui/url/abc.../detection",
"report_url": "https://urlscan.io/result/...",
"screenshot_url": "https://urlscan.io/screenshots/....png",
"score_breakdown": {
  "heuristics": 0, "whois": 0, "dns": 0.4, "ssl": 0, "redirects": 0,
  "virustotal": 0, "urlhaus": 0, "phishtank": 0, "gsb": 0,
  "combined_final": 0.4,
  "critical_signals_triggered": 0,
  "critical_sources": []
},
"ml_prediction": {
  "available": true,
  "prediction": 0,
  "ml_risk_level": "Safe",
  "phishing_probability": 3.2,
  "safe_probability": 96.8,
  "top_features": [...],
  "model_type": "RandomForestClassifier + CalibratedCV",
  "model_version": "2.0",
  "training_accuracy": 0.9312
},
"heuristics": { "flags": [], "flag_count": 0, "heuristic_score": 0, "entropy": 2.4 },
"whois": { "domain": "example.com", "domain_age_days": 10392, "registrar": "..." },
"dns": { "hostname": "example.com", "flags": [], "dns_score": 5, "details": {...} },
"ssl": { "hostname": "example.com", "flags": [], "ssl_score": 0, "details": {...} },
"redirects": { "hop_count": 1, "redirect_score": 0, "is_www_normalization": false },
"urlhaus": { "found": false, "urlhaus_score": 0 },
"phishtank": { "found": false, "feed_size": 300, "source": "openphish" },
"gsb": { "found": false, "api_available": true },
"all_flags": [],
"total_flags": 0
}

```

POST /scan/bulk

Scan multiple URLs simultaneously.

Request:

```
{
  "urls": [
    "https://google.com",
    "https://github.com",
    "http://suspicious.tk/login"
  ]
}
```

Limits: Maximum 10 URLs. Rate limit: 2 requests per minute.

Response (200 OK):

```
{
  "total_urls": 3,
  "completed": 3,
  "failed": 0,
  "results": [
    {
      "url": "https://google.com",
      "status": "complete",
      "risk_level": "Safe",
      "confidence_score": 0.4,
      "message": "SAFE ...",
      "total_flags": 1,
      "score_breakdown": {...}
    }
  ],
  "scan_duration_seconds": 43.67,
  "highest_risk_url": "http://suspicious.tk/login",
  "highest_risk_level": "High Risk"
}
```

POST /scan/async

Submit a URL for background scanning. Returns immediately with a job ID.

Request:

```
{ "url": "https://example.com" }
```

Response (200 OK — immediate):

```
{
  "job_id": "48815e55-bb89-4142-b025-10b258f4eb59",
  "url": "https://example.com",
  "status": "pending",
  "message": "Scan queued. Poll /scan/status/{job_id} for results.",
  "poll_url": "/scan/status/48815e55-bb89-4142-b025-10b258f4eb59",
}
```

```
"created_at": "2026-02-28T10:00:00"
}
```

GET /scan/status/{job_id}

Poll for the result of an async scan.

Job states: pending → running → complete or failed

Response when complete (200 OK):

```
{
  "job_id": "48815e55-...",
  "url": "https://example.com",
  "status": "complete",
  "progress_message": "Scan complete - Safe detected",
  "created_at": "2026-02-28T10:00:00",
  "started_at": "2026-02-28T10:00:00.021",
  "completed_at": "2026-02-28T10:01:05.040",
  "elapsed_seconds": 65,
  "result": { ...full ScanResult... },
  "error": null
}
```

Response when job not found (404):

```
{ "detail": "Scan job 'xxx' not found or expired." }
```

POST /feedback

Submit a correction to improve detection accuracy.

Request:

```
{
  "scan_id": "u-abc123...",
  "url": "https://example.com",
  "original_risk": "Medium Risk",
  "corrected_risk": "Safe",
  "feedback_type": "false_positive",
  "user_note": "This is our internal staging server",
  "confidence_score": 35.2,
  "total_flags": 4
}
```

feedback_type values: | Value | Meaning | |——-|———| | false_positive
| System said dangerous, URL is actually safe | | false_negative | System said

safe, URL is actually malicious | | **wrong_level** | Level is wrong (should be High, not Medium) | | **correct** | Result was accurate |

Response (200 OK):

```
{
  "feedback_id": 7,
  "status": "received",
  "message": "Feedback recorded. Thank you.",
  "submitted_at": "2026-02-28T10:05:00"
}
```

GET /feedback/stats

View feedback collection statistics.

Response (200 OK):

```
{
  "total_feedback": 12,
  "breakdown_by_type": {
    "false_positive": 3,
    "false_negative": 1,
    "wrong_level": 2,
    "correct": 6
  },
  "false_positives": 3,
  "false_negatives": 1,
  "recent_feedback": [...],
  "training_ready": false
}
```

training_ready: true when 50+ feedback samples collected — at this point the ML model should be retrained.

GET /metrics

Runtime performance metrics.

Response (200 OK):

```
{
  "summary": {
    "total_scans": 42,
    "cache_hit_rate_pct": 38.1,
    "cache_hits": 16,
  }
}
```



```

    "cache_misses": 26,
    "avg_scan_time_ms": 8234.5,
    "errors": 0
  },
  "risk_distribution": {
    "High Risk": {"count": 8, "percentage": 19.0},
    "Medium Risk": {"count": 5, "percentage": 11.9},
    "Low Risk": {"count": 3, "percentage": 7.1},
    "Safe": {"count": 26, "percentage": 61.9}
  },
  "threat_feed_hits": {
    "urlhaus": 3,
    "openphish": 1,
    "gsb": 0
  },
  "ml_predictions": {
    "total": 40,
    "high_risk": 9,
    "safe": 31
  },
  "daily_scans_last_7_days": {
    "2026-02-28": 42
  }
}

```

GET /metrics/prometheus

Same metrics in Prometheus text format for Grafana dashboards.

GET /proxy-image

Fetches an external screenshot image server-side and serves it to the browser.

Query parameter: `url` — full URL of the image to proxy

Use case: URLScan.io screenshots cannot be displayed directly in some environments due to CORS policy. This endpoint fetches the image from the Aegis server and returns the bytes to the client.

Example:

GET /proxy-image?url=https://urlscan.io/screenshots/019ca033-5a03-760a-8db9-2aa92df549df.png

Response: Raw image bytes with Content-Type: image/png

GET /health

Service health check.

```
{
  "status": "healthy",
  "ml_model": "loaded",
  "timestamp": "2026-02-28T10:00:00"
}
```

8. Scoring System (Technical)

Individual Layer Score Ranges

Layer	Max Score	Weight	Max Weighted
VirusTotal	100	30%	30.0
Heuristics	100	15%	15.0
WHOIS	100	15%	15.0
DNS	100	8%	8.0
SSL	100	7%	7.0
Redirects	100	10%	10.0
URLhaus	100	7%	7.0
OpenPhish	100	5%	5.0
Google Safe Browsing	100	3%	3.0
Total		100%	100.0

VirusTotal Score Mapping

$vt_score = \min(\text{malicious} \times 20 + \text{suspicious} \times 10, 100)$

Malicious	Suspicious	VT Score
0	0	0
1	0	20
3	0	60 → + override to 65 min
5	0	100 → + override to 90 min
8	3	100 → override fires

Critical Signal Definition

A “critical signal” is any single source with extreme confidence: - VirusTotal: malicious 3 (or malicious 1 AND suspicious 2) - Heuristics: score 60

- WHOIS: score 50 - DNS: score 40 - Redirects: score 50 - URLhaus, OpenPhish, GSB: score 80

When 3+ critical signals fire simultaneously, the URL is classified High Risk regardless of the weighted sum.

9. Data Storage & Privacy

What Is Stored

Data	Storage	TTL	Purpose
Scan results	Redis	1 hour	Caching to avoid re-scanning same URL
OpenPhish feed	Redis	6 hours	Avoid fetching on every scan
Rate limit counters	Redis	1 minute	Enforce per-minute rate limits
Scan results	Pinecone (vector DB)	Permanent	Semantic similarity search
Feedback submissions	SQLite	Permanent	ML retraining data
Metrics	Redis	30 days	Performance monitoring

What Is NOT Stored

- The content of the web pages scanned (we only analyse URLs and meta-data)
- User identity or authentication details (no user accounts in current version)
- Personal information from feedback submissions (only URL, risk level, feedback type)

External API Data Sharing

When a URL is submitted for scanning, it is sent to: - **VirusTotal** — URL is stored by VirusTotal and may be publicly visible on their platform - **URLScan.io** — Scan is created as “public” visibility by default; URL and screenshot are public - **Google Safe Browsing** — URL hash is sent for lookup; full URL is not transmitted - **abuse.ch (URLhaus)** — URL is sent for lookup - **OpenPhish** — No URL is sent; local feed is downloaded and matched client-side

10. Performance & Reliability

Scan Duration

Scenario	Typical Duration
Cache hit (previously scanned)	< 50ms
Full scan (first time)	60–90 seconds
Heuristics only	< 5ms
Phase 1 checks only (WHOIS, DNS, SSL, redirects)	3–15 seconds
VT + URLScan polling	60–90 seconds (dominates)

Concurrency

- All Phase 1 + Phase 2 checks run concurrently in a single `asyncio.gather()` call
- VT and URLScan submitted and polled concurrently
- Bulk scans: all URLs in the batch run concurrently — 10 URLs takes time of slowest single scan

Caching

Scan results are cached in Redis for 1 hour. The same URL submitted multiple times within 1 hour returns instantly from cache.

Rate Limiting

Endpoint	Limit
POST /scan	5 per minute
POST /scan/bulk	2 per minute
POST /scan/async	10 per minute

429 Too Many Requests is returned when limit is exceeded.

11. Configuration Reference

Environment Variables (.env file)

```
# VirusTotal - free account provides 4 lookups/minute
VT_API_KEY=your_virustotal_api_key
```

```

# URLScan.io - free account, register at urlscan.io
URLSCAN_API_KEY=your_urlscan_api_key

# Pinecone - vector database for scan memory
PINECONE_API_KEY=your_pinecone_api_key

# Google Safe Browsing - enable in Google Cloud Console
GSB_API_KEY=your_google_cloud_api_key

# URLhaus / abuse.ch - free registration at auth.abuse.ch
URLHAUS_API_KEY=your_urlhaus_api_key

# Ollama host (if using local LLM integration)
OLLAMA_HOST=http://host.docker.internal:11434

# Paths (set automatically by docker-compose)
FEEDBACK_DB_PATH=/code/app/data/feedback.db
ML_MODEL_PATH=/code/app/ml/model.pkl

```

Docker Services

Service	Port	Purpose
link-analyzer	8000	FastAPI application
redis	6379	Caching and rate limiting

12. Limitations & Known Issues

Detection Limitations

False positives (safe sites flagged): | Site type | Why flagged | Impact |
 |————|————|———| | New legitimate sites | Domain age < 30 days | Low
 — WHOIS score alone can't classify High Risk | | Sites using Let's Encrypt |
 Free CA flag fires | Low — 10 SSL score points only | | Product sites without
 email | dns_no_mx flag fires | Low — DNS score partial contributor | | Sites with
 long URLs | Length heuristic fires | Low — 5 points only |

False negatives (malicious sites not caught): | Scenario | Why missed
 | Mitigation | |————|————|————| | Brand-new phishing URL, zero VT
 detections | Not yet in any threat database | Heuristics, WHOIS, DNS still detect
 structural signals | | Compromised legitimate domain | Domain is old and trusted
 | VT will eventually flag; GSB covers many | | Private/internal malicious URL
 | Not indexed by public feeds | Heuristics and structural analysis still apply |

Known False Flags

URL	Flag	Status
claude.ai	No MX records	Expected — product site, not email domain
claude.ai	Let's Encrypt flag	Expected — legitimate use of free CA
github.com	No MX records	Expected — github.com is not an email domain

Technical Limitations

Limitation	Detail
Scan time	60–90 seconds per URL (dominated by VT/URLScan polling)
URL shorteners	Can only analyse the destination after following the chain
JavaScript-heavy pages	Redirect analysis uses HTTP headers only; JS redirects not followed
Authenticated pages	Cannot scan behind login walls
Rate limits	Free tier API keys have daily/hourly limits
ML training data	Currently trained on URL structure features primarily; improves significantly with real scan data feedback

13. FYP Summary

Project Information

Project Title: Aegis AI — Intelligent URL Threat Analysis Service

Academic Context: Final Year Project (FYP) — Computer Science / Cyber-security

Development Duration: 4 phases over the project period

Stack: Python 3.11, FastAPI, Redis, Pinecone, SQLite, scikit-learn, Docker

Feature Completion

Phase	Features	Status
Phase 1 — Local Intelligence	URL Heuristics, WHOIS, DNS, SSL, Redirect Analysis	Complete
Phase 2 — Threat Intelligence	URLhaus, OpenPhish, Google Safe Browsing	Complete
Phase 3 — Architecture	Bulk Scan, Async Scan, Feedback Loop	Complete
Phase 4 — Machine Learning	Feature Extraction, ML Classifier, Metrics	Complete

Key Innovations

1. Multi-layer detection with weighted fusion Rather than binary block/allow decisions, Aegis assigns weighted scores from 11 independent sources and fuses them mathematically — enabling nuanced risk levels that reflect actual confidence.

2. Critical signal override system Pure mathematical weighting can underclassify URLs that score high on only one or two dimensions (e.g., a URL with 8 VT detections but zero structural signals). The override system guarantees minimum scores when any single source has extreme confidence.

3. Explainable ML predictions The ML classifier reports not just a prediction but the top features that drove it, with their exact contribution scores. This makes the AI decision auditable and explainable — a critical requirement in security applications.

4. Feedback loop for continuous improvement Every false positive or false negative can be submitted as labeled training data. When 50+ corrections accumulate, the ML model can be retrained with the corrected labels weighted 5× more than base training data.

5. Graceful degradation Every layer is wrapped in exception handling with safe defaults. If URLhaus is unreachable, the scan continues. If the ML model file isn't present, the scan continues. If Redis is unavailable, the scan continues without caching. No single point of failure.

API Endpoints Summary

Endpoint	Method	Purpose
/scan	POST	Full synchronous URL scan
/scan/bulk	POST	Scan up to 10 URLs simultaneously

Endpoint	Method	Purpose
/scan/async	POST	Non-blocking background scan
/scan/status/{id}	GET	Poll async scan result
/feedback	POST	Submit result correction
/feedback/stats	GET	View feedback statistics
/metrics	GET	Service metrics (JSON)
/metrics/prometheus	GET	Metrics (Prometheus format)
/proxy-image	GET	Serve external screenshot images
/health	GET	Service health check
/	GET	Service information

Results

Detected URLs: - <http://paypal-secure-verify-account.tk/login/confirm>
→ High Risk 90%, ML 85.27%, 12 flags, 8/94 VT engines - <https://google.com>
→ Safe 0.4%, ML 3.2%, 0 critical flags - <https://github.com> → Safe 0.8%,
0 critical flags - <https://claude.ai> → Safe 2.7%, 3 minor flags (known
limitations documented)

Concurrent bulk scan: 3 URLs processed in 43.67 seconds (all concurrent)

Async scan: Job submitted in < 100ms, result retrieved by polling

Document version: 4.0 — Last updated: February 2026